

Sistemas inteligentes para radiocomunicaciones con SDR

Entornos de desarrollo para Python y SDR

Dr. Yanqueleth Molina-Tenorio

UAM Iztapalapa
PCyTI

Trimestre 26-I

Entorno de desarrollo

Un entorno de desarrollo es el espacio de trabajo digital donde todo está listo para programar, ejecutar, depurar y mantener un proyecto.

Elementos principales

- IDE o editor de código
- Intérprete de Python
- Módulos y librerías
- Dependencias del sistema
- Drivers y controladores
- Documentación y scripts

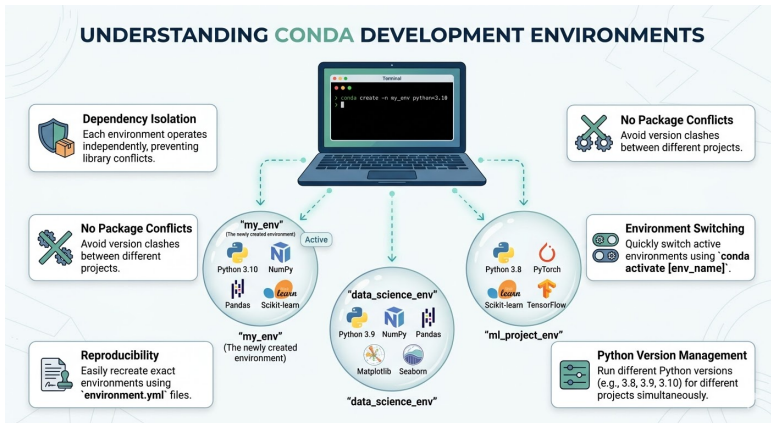
Para proyectos con SDR

- Acceso al hardware desde Python
- Instalación consistente de bibliotecas
- Manejo de versiones por proyecto
- Reproducibilidad experimental
- Integración con adquisición en tiempo real
- Menos conflictos entre paquetes

Ventajas de un entorno bien configurado

- **Aislamiento de versiones:** cada proyecto puede usar librerías distintas sin interferencias.
- **Reproducibilidad:** otro colega puede reconstruir el entorno con un archivo como `requirements.txt`.
- **Compatibilidad con hardware:** facilita trabajar con drivers, bindings y SDKs específicos para SDR.
- **Ahorro de tiempo:** evita reinstalar y reconfigurar herramientas cada vez que se inicia un proyecto.
- **Escalabilidad:** permite crecer hacia adquisición en tiempo real, visualización, automatización e IA.
- **Mantenimiento:** documenta qué se instaló, por qué se instaló y cómo volver a usarlo.

Idea clave: un buen entorno de desarrollo reduce errores técnicos y deja más tiempo para enfocarse en el análisis de la señal.



Preparación del entorno Python para control de SDRs

Objetivo del entorno:

- Instalar y cargar las librerías necesarias.
- Configurar los módulos y dependencias del proyecto.
- Asegurar que Python pueda reconocer el hardware SDR.

```
python3 -m venv sdr_course  
source sdr_course/bin/activate  
python -m pip install --upgrade pip  
pip install numpy scipy matplotlib jupyter pyqtgraph
```

Paquetes según el dispositivo:

- pyrtlsdr para RTL-SDR
- SoapySDR cuando se trabaja con una capa de abstracción común
- pyadi-iio para ADALM-Pluto

Buenas prácticas para entornos de desarrollo

- 1 Crear un entorno virtual por proyecto.
- 2 Nombrar claramente el entorno, por ejemplo `sdr_course` o `pluto_rx_lab`.
- 3 Guardar dependencias con:

```
pip freeze > requirements.txt
```

- 4 Verificar desde Python que el hardware sea visible antes de programar tareas complejas.
- 5 Separar adquisición, procesamiento y visualización en scripts o módulos distintos.
- 6 Documentar versión de Python, sistema operativo y SDR utilizado.

Errores comunes a evitar

- Instalar librerías globalmente y mezclar proyectos distintos.
- No documentar dependencias ni versiones.
- Probar código sin validar primero que el SDR fue detectado por el sistema.
- Confundir problemas de software con problemas de drivers o permisos del dispositivo.
- Querer depurar adquisición, procesamiento e interfaz gráfica al mismo tiempo.

Recomendación: primero validar hardware, luego lectura de muestras, después procesamiento y finalmente visualización en tiempo real.

Practica: Configuración del entorno de desarrollo SDR

Objetivo

Configurar un entorno funcional para adquisición y procesamiento SDR en Python.

1. Entorno base

- Instalar Anaconda / Miniconda
- Crear entorno

2. Librerías

- SoapySDR
- SDRRTL

3. Drivers SDR

- RTL-SDR
- MiniLime SDR

4. Verificación

- `SoapySDRUtil -find`
- ```
from SoapySDR import
print(Device.enumerate())
```

## Actividad

Crear entorno, instalar drivers, verificar SDR y ejecutar script básico.

Nota: Manual de instalación